RAG SENSE

DAY 1 — IMPLEMENTATION NOTES (PART 1)

Project setup • document ingestion • chunking • embeddings • ChromaDB • document IDs • retrieval foundations

Purpose of this document

This is a detailed checkpoint of everything covered so far in Day 1. It records the design decisions, tested components, code patterns, commands, limitations, and interview questions. It intentionally stops at the point reached: the basic similarity-search test has been prepared, but its output and retrieval-quality review are still pending.

Important: This is Part 1, not a declaration that Day 1 is complete. After reviewing these notes, continue with the remaining Day 1 retrieval work.

Development environment: Windows • VS Code • PowerShell • Python 3.12 • 16 GB RAM

1. Project overview and architecture

RAG Sense is an end-to-end Retrieval-Augmented Generation (RAG) application. Its purpose is to let a user upload documents, extract their text, split that text into manageable chunks, convert chunks into vectors (embeddings), store those vectors, and retrieve relevant chunks when a question is asked.

The pipeline being implemented in Day 1 is the retrieval foundation. The language model / answer-generation stage comes after retrieval is reliable.

```
User document
    |
    v
Document Loader
(PDF / DOCX / TXT / MD / CSV / XLSX / PPTX / JSON / HTML)
    |
    v
LangChain Document objects
(page_content + metadata)
    |
    v
Text Chunking
(RecursiveCharacterTextSplitter)
    |
    v
Embedding Model
(BAAI/bge-small-en-v1.5, 384 dimensions)
    |
    v
ChromaDB Vector Store
(persistent local collection)
    |
    v
User question -> query embedding
    |
    v
Similarity Search -> top-K relevant chunks
    |
    v
[Later] Context construction -> LLM -> grounded answer
```

What RAG means

Retrieval-Augmented Generation combines retrieval from an external knowledge source with generation by a language model. Instead of asking an LLM to answer only from its learned parameters, the application first retrieves relevant document passages and then supplies those passages as context to the LLM.

• **Retrieval:** find relevant passages from the user's documents.

• **Augmentation:** add the retrieved passages to the prompt/context.

• **Generation:** ask the LLM to answer using that context.

Current Day 1 boundary

Implemented/tested so far: project environment and Git setup, document loading, chunking, local embeddings, ChromaDB initialization, and insertion of a large sample document. The basic search function has been added and a test script was provided, but the user has not yet shared the search output. Similarity-score inspection, retrieval quality checks, metadata filtering, and a full end-to-end verification remain to be completed.

2. Environment and Git setup

Project location and structure

```
D:\development\RAG_SENSE\
■■■ Backend\
    ■■■ app\
    ■   ■■■ __init__.py
    ■   ■■■ ingestion\
    ■   ■   ■■■ __init__.py
    ■   ■   ■■■ loader.py
```

```
■   ■   ■■■ chunker.py
■   ■■■ embeddings\
■   ■   ■■■ __init__.py
■   ■   ■■■ embedding_model.py
■   ■■■ vectorstore\
■       ■■■ __init__.py
■       ■■■ chroma_store.py
■■■ data\
■   ■■■ sample files (local test data)
■■■ venv\
■■■ requirements.txt
■■■ test_loader.py
■■■ test_chunk.py
■■■ test_embeddings.py
■■■ test_chroma.py
■■■ test_add_documents.py
■■■ test_document.py
```

The exact current structure may also include `test_search.py` if you have created the search test file.

Virtual environment

```
# From the Backend directory
python -m venv venv

# Activate in PowerShell
.\venv\Scripts\Activate.ps1

# Install project dependencies
pip install -r requirements.txt
```

A virtual environment isolates this project's Python packages from other Python projects. Activate it before running tests or installing packages.

Requirements used

```
langchain
langchain-community
langchain-chroma
langchain-huggingface
chromadb
sentence-transformers
pypdf
python-docx
docx2txt
pandas
openpyxl
python-pptx
beautifulsoup4
python-dotenv
unstructured
langchain-text-splitters
```

Install the PPTX dependency using **python-pptx**; the Python import is `from pptx import Presentation`. The package name is not `presentation`.

GitHub push issue and fix

The first push was rejected because a file inside `Backend/venv` was a 114.79 MB llvmlite DLL, exceeding GitHub's 100 MB file limit. A virtual environment should never be committed.

```
git rm -r --cached Backend/venv
git add .gitignore
git commit --amend --no-edit
git push -u origin main
```

The push then succeeded. The root `.gitignore` contains:

```
Backend/venv/
Backend/.env
.env
Backend/chroma_db/
Backend/data/
__pycache__/
*.pyc
.vscode/
```

Keep API keys, environment files, local databases, test documents, virtual environments, and generated caches out of Git. Verify the ignore rules before pushing.

3. Document ingestion / loading

A loader reads a file and converts its content into LangChain `Document` objects. A Document typically has two important fields:

• `page_content`: the extracted text.

• `metadata`: information about the source, page, sheet, slide, or other origin.

```
from langchain_core.documents import Document

example = Document(
    page_content="Python is a programming language.",
    metadata={"source": "sample.pdf", "page": 0}
)
```

Metadata is important because later the application may need to show citations, identify the source file, filter by user/document, or delete all chunks belonging to a document.

Formats covered in the loader

File type	Loading approach / package	Notes
PDF	PyPDFLoader (pypdf)	Page-based Documents; scanned PDFs need OCR.
DOCX	Docx2txtLoader	Extracts document text.
TXT / MD	TextLoader	Plain text / Markdown content.
CSV	CSVLoader	Rows can be represented as Documents.
XLSX	pandas + openpyxl	Sheets can be loaded separately; sheet name in metadata.
PPTX	python-pptx	Extracts slide text and tables; embedded images are not OCR'd.
JSON	Python JSON parser	Convert structured content into readable text.
HTML / HTM	BSHTMLLoader / BeautifulSoup	Extracts page text; clean boilerplate where needed.

Loader function and test pattern

The loader function used in the project is named `load_document`. The test pattern is:

```
from app.ingestion.loader import load_document

documents = load_document("data/sample.pdf")

print("Documents loaded:", len(documents))
print("First page content:", documents[0].page_content[:500])
print("Metadata:", documents[0].metadata)
```

Run the test from the Backend directory:

```
python .\test_loader.py
```

The actual loader module contains the format-specific branching. The code above shows the public function and how it is called; it is not a replacement for the complete loader implementation.

Ingestion limitations to remember

• A scanned PDF is usually just page images. Text extraction needs an OCR step such as Tesseract or a cloud OCR service.

• PPTX text extraction does not automatically read text inside pictures or screenshots.

• Tables, headers, footers, complex layouts, and unusual encodings may need custom handling.

• Always preserve source metadata. For a PDF, page metadata is useful for citations; for spreadsheets, sheet names are useful.

4. Chunking

Large documents are split into smaller text sections called **chunks**. Embedding an entire large document as one vector can dilute topic-specific meaning and makes retrieval less precise. Chunking allows the retriever to return a focused passage.

RecursiveCharacterTextSplitter

The project uses LangChain's `RecursiveCharacterTextSplitter`. It tries larger natural boundaries first, such as paragraph breaks, then line breaks, spaces, and finally characters.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

def chunks_document(documents):
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200,
        length_function=len,
        separators=["\\n\\n", "\\n", " ", ""]
    )
    return text_splitter.split_documents(documents)
```

Important exact function name: `chunks_document` (singular “document” after “chunks”). Use this exact name in imports and tests.

Chunk settings

• **chunk_size=1000:** target maximum chunk length of roughly 1,000 characters because `length_function=len` counts characters, not tokens.

• **chunk_overlap=200:** adjacent chunks share approximately 200 characters, helping preserve context near boundaries.

• **length_function=len:** defines how chunk length is measured.

• **separators:** paragraph, newline, space, and character boundaries are tried in order.

Overlap helps when an important sentence or concept falls across a chunk boundary. Too much overlap increases storage and repeated content; chunks that are too small can lose context, while chunks that are too large can retrieve irrelevant surrounding text.

Chunking test

```
from app.ingestion.loader import load_document
from app.ingestion.chunker import chunks_document

documents = load_document("data/sample.pdf")
chunks = chunks_document(documents)

print("Original documents/pages:", len(documents))
print("Total chunks:", len(chunks))
print("First chunk:", chunks[0].page_content[:500])
print("First chunk metadata:", chunks[0].metadata)
```

The sample PDF produced approximately 295 page-level Documents and 1,471 chunks in the recorded test. Exact counts depend on the input file and splitter settings.

```
python .\test_chunk.py
```

5. Embeddings

An embedding model converts text into a numeric vector. Texts with related meanings are expected to have vectors that are close under the vector database's distance metric. The model makes semantic retrieval possible; it is not itself

the vector database.

Selected model

BAAI/bge-small-en-v1.5 was selected after the hosted Hugging Face inference attempt ran out of credits (HTTP 402). BGE-M3 was also tried, but its download/resource cost was too high for the current development pace. The selected small model produces 384-dimensional vectors and is English-focused.

```
from langchain_huggingface import HuggingFaceEmbeddings

def get_embedding_model():
    return HuggingFaceEmbeddings(
        model_name="BAAI/bge-small-en-v1.5",
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True}
    )
```

Embedding test

```
from app.embeddings.embedding_model import get_embedding_model

embeddings = get_embedding_model()

texts = [
    "Python is a programming language.",
    "Machine learning uses algorithms to learn from data.",
    "[illegible]"
]

vectors = embeddings.embed_documents(texts)
print("Embedding generation successful!")
print("Number of embeddings:", len(vectors))
print("Vector dimensions:", len(vectors[0]))

query_vector = embeddings.embed_query("What is Python?")
print("Query vector dimensions:", len(query_vector))
```

Expected dimensions: **384** for each vector. Run:

```
python .\test_embeddings.py
```

Why the same embedding model must be used

Documents are embedded when they are inserted. A user's query is embedded at search time. Those vectors must be generated in the same compatible vector space. If you index with one model and query with an unrelated model, vector comparisons are not meaningful.

Language support caveat

Passing Hindi text into an English-focused model does not prove that the model supports Hindi semantic retrieval well. RAG Sense has a multilingual ambition, but the currently selected BGE Small model does not fully satisfy that requirement. Treat multilingual embedding support as a future model/provider decision and test retrieval in every target language before claiming it works.

Local model loading and HF warning

• The model weights are downloaded and cached on the machine the first time. Later processes normally reuse the cache rather than downloading the weights again.

• A new Python process may still load the cached model weights into RAM again. That is normal initialization, not necessarily a fresh download.

• The unauthenticated Hugging Face Hub warning is generally a rate-limit notice, not an embedding failure. A token can help with download limits, but do not commit it to Git.

• The model runs on CPU in the current configuration. CPU embedding of 1,471 chunks can take noticeable time.

6. ChromaDB vector store

ChromaDB is a vector database. It stores embeddings alongside the original text and metadata so that the application can retrieve relevant chunks by vector similarity.

ChromaDB versus MongoDB

MongoDB	ChromaDB
General-purpose document database.	Vector database designed for embedding storage and similarity retrieval.
Stores application records/documents and supports queries/indexes.	Stores vectors, text/documents, metadata, and supports nearest-neighbour search
Can store users, profiles, chats, permissions, and application data.	Can store searchable document chunks and their embeddings.

They are not direct substitutes. A larger RAG product may use MongoDB (or SQL) for users and application records, and ChromaDB for vector retrieval.

Current Chroma store pattern

```
from langchain_chroma import Chroma
import hashlib
from app.embeddings.embedding_model import get_embedding_model

# Lazy initialization pattern (if used in your current file)
embedding_function = None

def get_vector_store():
    global embedding_function

    if embedding_function is None:
        embedding_function = get_embedding_model()

    vector_store = Chroma(
        collection_name="rag_sense_documents",
        embedding_function=embedding_function,
        persist_directory="./chroma_db"
    )
    return vector_store
```

The current code has evolved during implementation. Compare this pattern with your actual `chroma_store.py`; keep the lazy-loading version if that is what you have tested. The important configuration is the collection name, embedding function, and persistent directory.

• **collection_name:** logical collection inside Chroma, similar to a named group of vector records.

• **embedding_function:** converts document text and queries into vectors.

• **persist_directory:** stores the local Chroma database on disk. From the Backend working directory, `./chroma_db` resolves to `Backend/chroma_db`.

• Chroma creates the directory automatically. With persistent configuration, it saves data automatically; a separate `persist()` call is not required in this LangChain setup.

Initialization test

```
from app.vectorstore.chroma_store import get_vector_store

vector_store = get_vector_store()

print("ChromaDB initialized successfully!")
print("Collection:", vector_store._collection.name)
print("Existing documents:", vector_store._collection.count())
```

The recorded initialization test succeeded and initially showed the collection `rag_sense_documents` with zero stored records.

Persistence: local versus cloud

Local persistence means the vectors are saved to a directory on the development machine. A cloud-hosted Chroma service or a self-hosted persistent server is a separate deployment architecture. A local folder on a developer's

computer is not automatically a cloud database.

Render Free deployment warning: free web services have limited memory (512 MB), spin down, and do not provide a persistent disk by default. Files written to the service's ephemeral filesystem can be lost on restart/redeploy. Do not rely on local `chroma_db` or uploaded files there as permanent storage. A production deployment will need an external persistent vector database/storage and a suitable embedding strategy. BGE Small may also be too resource-heavy for a 512 MB service.

7. Adding chunks to ChromaDB and duplicate prevention

The insert operation stores chunk text, metadata, and generated embeddings. LangChain's Chroma integration calls the configured embedding model for the documents.

Stable IDs and current approach

A key design question is how to identify each chunk. Random IDs create a new record every time the same document is re-ingested. Deterministic IDs make repeat ingestion idempotent for the same source, content, and chunk order.

```
import hashlib

def make_chunk_id(source, page_content, index):
    content_hash = hashlib.sha256(
        page_content.encode("utf-8")
    ).hexdigest()[:12]

    raw_id = f"{source}_{content_hash}_{index}"
    return hashlib.sha256(raw_id.encode("utf-8")).hexdigest()
```

This is the ID-generation logic used in the current implementation pattern. It includes source, a hash of chunk content, and chunk index, then hashes the combined string to produce a Chroma-safe ID.

Batch insertion pattern

```
def add_documents(vector_store, documents, batch_size=100):
    if not documents:
        print("No documents to add.")
        return

    total = len(documents)

    for start in range(0, total, batch_size):
        batch = documents[start:start + batch_size]
        ids = []

        for index, document in enumerate(batch, start=start):
            source = document.metadata.get("source", "unknown")

            content_hash = hashlib.sha256(
                document.page_content.encode("utf-8")
            ).hexdigest()[:12]

            chunk_id = f"{source}_{content_hash}_{index}"
            chunk_id = hashlib.sha256(
                chunk_id.encode("utf-8")
            ).hexdigest()

            ids.append(chunk_id)

        vector_store.add_documents(
            documents=batch,
            ids=ids
        )

        print(
            f"Processed {min(start + batch_size, total)}/{total} chunks",
            flush=True
        )

    print("All chunks processed successfully!")
```

The user confirmed the batch-insertion approach was acceptable, but the exact latest contents of the file have not been rechecked. Ensure `hashlib` is imported at the top if this code is used.

What happens with duplicate filenames?

• Two different files can have the same filename. If their chunk contents differ, their content hashes differ, so their IDs will normally differ.

• If the same file is ingested again with the same source, same chunk content, and same chunk order, the deterministic IDs are repeated. This avoids creating a second set of records when the vector store's add/upsert behavior handles the same IDs.

• If the content changes, new IDs can be created. Old chunks may remain unless the old document's chunks are explicitly deleted or replaced.

• A filename is not a robust identity for a document. Two users may upload files with the same name, and a user may upload a changed version under the same name.

Recommended production identity design — planned, not implemented yet

For a multi-user application, introduce a generated document UUID and a user ID. Store these in metadata for every chunk.

```
# Conceptual metadata for every chunk
{
    "user_id": "authenticated-user-uuid",
    "document_id": "generated-document-uuid",
    "source": "annual_report.pdf",
    "chunk_index": 12,
    "page": 4
}
```

A stronger chunk identity can be based on `document_id + chunk_index` (or a stable hash of those values). The document UUID distinguishes two uploads even if their filenames are identical. The user ID supports per-user filtering and access control.

Do not confuse this recommendation with the current implementation: the current code uses source/content-hash/index IDs. UUID-based document management, user isolation, document-version replacement, and deletion have not yet been confirmed as implemented.

Why metadata filtering matters

In a multi-user system, a query must not retrieve another user's private document chunks. Retrieval should filter by the authenticated user's ID, and the server must derive that ID from the authenticated session/token—not trust a user_id sent freely by the browser.

```
# Conceptual example; exact syntax depends on the installed Chroma/LangChain version
results = vector_store.similarity_search(
    query,
    k=5,
    filter={"user_id": authenticated_user_id}
)
```

Metadata filtering is not a replacement for authorization checks, but it is an important part of separating users' retrieval results.

Updating and deleting documents

• When a document is deleted, find all chunks with its document_id and delete those vector records.

• When a document is replaced, remove the previous version's chunks and insert the new version, or keep version IDs and mark which version is active.

• Keep a document registry in an application database (for example, SQLite during local development or a production database later) with document_id, user_id, filename, status, timestamps, and storage location.

• The current Day 1 code has not yet implemented this lifecycle.

Recorded insertion result

The user successfully inserted the sample PDF's chunks. The terminal reported approximately: **Successfully processed 1471 chunks** and **Total stored chunks: 1471**. This confirms the local embedding + Chroma insertion path worked for that test document.

8. Similarity search — current next step

Similarity search retrieves the top-K chunks whose vectors are closest to the query vector. The vector store automatically embeds the query using the same configured embedding function.

```
def search_documents(vector_store, query, k=5):
    results = vector_store.similarity_search(
        query=query,
        k=k
    )
    return results
```

This function has been added according to the user's confirmation.

Test script

```
from app.vectorstore.chroma_store import (
    get_vector_store,
    search_documents
)

vector_store = get_vector_store()

query = "What is Python used for?"

results = search_documents(
    vector_store,
    query,
    k=5
)

print("\\nRetrieved Documents:\\n")

for index, result in enumerate(results, start=1):
    print(f"\\nResult {index}")
    print("Content:", result.page_content)
    print("Metadata:", result.metadata)

# Run from Backend
python .\test_search.py
```

Important test condition: use a question that is actually answered by the sample PDF. If the sample PDF is not about Python, a Python query may return unrelated chunks. That would not necessarily mean the search code is broken.

The next action is to run this test and inspect the returned content and metadata. Do not mark similarity search complete until the output is reviewed.

9. Next Day 1 tasks — continue from here

• **Task 1 — Run basic retrieval:** execute `test_search.py` using a query grounded in the sample PDF. Check whether the top 5 chunks are relevant.

• **Task 2 — Inspect distance:** add a test using `similarity_search_with_score`. In the common Chroma/LangChain distance output, lower distance generally means closer/more similar; it is not a percentage and should not be described as a higher-is-better similarity score.

• **Task 3 — Test persistence:** close the Python process, start a new process, reopen the vector store, and verify the collection count remains 1,471 (assuming no new insertions/deletions).

• **Task 4 — Test duplicate ingestion:** run ingestion again with the same stable IDs and verify that the collection does not grow by another full 1,471 records. Confirm the exact add/upsert semantics in the installed version.

• **Task 5 — Inspect metadata:** confirm source/page/sheet/slide metadata survives loading, chunking, insertion, and retrieval.

• **Task 6 — Metadata filtering:** add and test a document/user filter as a retrieval concept; production user isolation is still future work.

• **Task 7 — End-to-end Day 1 check:** file -> loader -> chunks -> embeddings -> Chroma -> query -> relevant chunks.

• **Task 8 — Ask for Day 1 notes:** once all remaining tasks are done, request the final complete Day 1 notes/PDF update. Add the score, filter, test outputs, and any code changes from the remaining session.

10. Interview questions and answers

RAG and ingestion

Q. What is RAG?
Answer: Retrieval-Augmented Generation retrieves relevant information from an external knowledge source and supplies it as context to an LLM before generating an answer.

Q. Why do we need RAG if an LLM already has knowledge?
Answer: An LLM may not know private, recent, or domain-specific documents. RAG lets the application retrieve that information at query time and ground the answer in it.

Q. What are the three main stages of RAG?
Answer: Retrieval, augmentation (adding retrieved context to the prompt), and generation.

Q. What is a LangChain Document?
Answer: A structured object containing page_content and metadata. The text is used for chunking/embedding; metadata records the source and other context.

Q. Why preserve metadata during ingestion?
Answer: It supports citations, source display, filtering, access separation, debugging, and document-level deletion.

Q. Why does a scanned PDF need OCR?
Answer: A scanned PDF may contain page images rather than selectable text. A normal PDF text loader cannot extract text that is not encoded as text.

Q. Does the PPTX loader extract text inside images?
Answer: Not with the current text-and-table extraction approach. That requires OCR or an image-understanding step.

Chunking

Q. Why split a document into chunks?
Answer: Smaller passages make embedding and retrieval more focused, avoid embedding huge documents as one unit, and allow the system to return only relevant context.

Q. Why use RecursiveCharacterTextSplitter?
Answer: It tries natural text boundaries in order, helping avoid cutting paragraphs or sentences unnecessarily.

Q. What does chunk_size=1000 mean here?
Answer: The splitter targets 1,000 characters because length_function is len. It does not mean 1,000 tokens.

Q. Why use chunk_overlap=200?
Answer: It preserves some context between adjacent chunks so information near a boundary is less likely to be lost.

Q. What happens if chunks are too small?
Answer: They may lack context and produce fragmented retrieval.

Q. What happens if chunks are too large?
Answer: They can contain unrelated material, reduce retrieval precision, and use more context when passed to the LLM.

Q. What is the role of metadata in chunking?
Answer: Chunking normally preserves the original Document metadata, so each chunk can still be traced to its source/page/sheet.

Embeddings and vector search

Q. What is an embedding?
Answer: A numeric vector representing semantic features of text. Similar meanings are expected to be close in vector space.

Q. What is the dimension of the selected BGE Small embedding?
Answer: 384 dimensions.

Q. Why use the same embedding model for documents and queries?
Answer: Both must be represented in the same compatible vector space for distance comparisons to be meaningful.

Q. Why did we choose BGE Small?
Answer: It is relatively lightweight and practical for local CPU development compared with the larger BGE-M3 option. The hosted HF inference attempt was blocked by exhausted credits.

Q. Does BGE Small meet the multilingual requirement?
Answer: Not fully. The selected model is English-focused. Multilingual retrieval must be addressed by choosing a suitable multilingual model/provider and evaluating it with target-language queries.

Q. What is the difference between embedding and a vector database?
Answer: The embedding model converts text to vectors; the vector database stores those vectors and retrieves nearest matches.

Q. What is top-K retrieval?
Answer: It returns the K closest matching chunks for a query. For example, k=5 asks for the five best matches.

ChromaDB, persistence, and identity

Q. Why ChromaDB instead of MongoDB?
Answer: Chroma is specialized for vector similarity search. MongoDB is a general-purpose database. They can complement each other rather than replace each other.

Q. What does persist_directory do?
Answer: It configures a local on-disk Chroma store so collection data can be reopened across Python processes on the same machine.

Q. Does Chroma automatically create the local directory?
Answer: Yes, the persistent client integration creates the directory when needed.

Q. Why should we not store the virtual environment in Git?
Answer: It is large, machine-specific, reproducible from requirements, and may contain files that exceed GitHub's file-size limit.

Q. Why are deterministic chunk IDs useful?
Answer: They help make repeated ingestion of the same source/chunk stable and prevent creating a fresh duplicate set on every run.

Q. What is the weakness of using only a filename as identity?
Answer: Different users can upload files with the same name, and a changed file can reuse the same name. A filename is not a globally unique document identity.

Q. Why introduce document_id as a UUID?
Answer: It uniquely identifies an upload independently of filename and allows all chunks from that upload to be grouped, replaced, or deleted.

Q. Why store user_id in chunk metadata?
Answer: It allows retrieval to filter results to the authenticated user's documents and helps prevent cross-user data leakage.

Q. Can the frontend be trusted to send the correct user_id?
Answer: No. The backend should derive the user identity from the authenticated session/token and apply the filter server-side.

Q. What if a document is re-uploaded with changed content?
Answer: The application should treat it as a new version or replace the previous document's chunks. Otherwise old chunks may remain and appear in retrieval.

Q. Why is Render Free not suitable for relying on local Chroma persistence?
Answer: The service has limited memory and no persistent disk by default; local filesystem data can be lost after restarts or redeploys. Use external persistent storage for production.

Explain the current project in an interview

"In the first stage of RAG Sense, I built a document ingestion and retrieval pipeline. I load multiple document formats into LangChain Document objects while preserving source metadata. I split the content using RecursiveCharacterTextSplitter with a 1,000-character chunk size and 200-character overlap. I generate 384-dimensional embeddings locally using BAAI/bge-small-en-v1.5 and store the chunks and vectors in a persistent ChromaDB collection. I also added deterministic chunk IDs based on source, content hash, and chunk index to reduce duplicate insertion. The next stage is validating top-K similarity retrieval, distances, metadata filtering, and robust document/user lifecycle management."

Be precise in an interview: UUID-based document management, production multi-user isolation, multilingual retrieval, and cloud persistence are design improvements still to be implemented—not completed features.

11. Quick revision sheet

• **Loader:** file -> LangChain Documents.

• **Document:** page_content + metadata.

• **Chunking:** RecursiveCharacterTextSplitter; 1,000 characters; 200 overlap.

• **Exact function:** chunks_document.

• **Embedding model:** BAAI/bge-small-en-v1.5; CPU; normalized embeddings; 384 dimensions.

• **Language caveat:** English-focused; multilingual quality not established.

• **Vector store:** ChromaDB collection rag_sense_documents.

• **Persistence path:** ./chroma_db relative to Backend working directory.

• **Insertion:** deterministic source + content hash + index IDs; batch size pattern 100.

• **Recorded insert result:** 1,471 chunks.

• **Search function:** search_documents(vector_store, query, k=5).

• **Not yet verified:** search output, score interpretation, persistence restart test, duplicate test, metadata filters, full Day 1 completion.

End of Part 1. Continue with Section 9 in the next working session and update these notes with the actual retrieval test results.